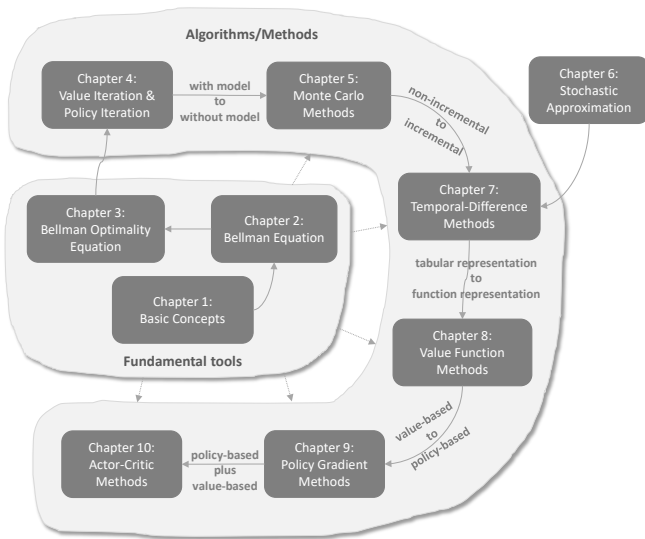


Lecture 5: Monte Carlo Methods

Shiyu Zhao

Department of Artificial Intelligence
Westlake University



- 1 动机示例
- 2 最简单的基于蒙特卡洛的强化学习算法
 - 算法: MC Basic
- 3 更高效地利用数据
 - 算法: MC Exploring Starts
- 4 无需探索性起始的蒙特卡洛方法
 - 算法: MC ϵ -贪婪

- 1 动机示例
- 2 最简单的基于蒙特卡洛的强化学习算法
 - 算法: MC Basic
- 3 更高效地利用数据
 - 算法: MC Exploring Starts
- 4 无需探索性起始的蒙特卡洛方法
 - 算法: MC ϵ -贪婪

动机示例：蒙特卡洛估计

▷ 如何在**没有模型**的情况下估计某些量？

- 最简单的想法：蒙特卡洛估计。

▷ 示例：抛硬币

结果（正面或反面）记为随机变量 X

- 若结果为正面，则 $X = +1$
- 若结果为反面，则 $X = -1$

目标是计算 $\mathbb{E}[X]$ 。

▷ 如何在**没有模型**的情况下估计某些量？

- 最简单的想法：蒙特卡洛估计。

▷ **示例：抛硬币**

结果（正面或反面）记为随机变量 X

- 若结果为正面，则 $X = +1$
- 若结果为反面，则 $X = -1$

目标是计算 $\mathbb{E}[X]$ 。

▷ 方法一：基于模型

- 假设概率模型已知为

$$p(X = 1) = 0.5, \quad p(X = -1) = 0.5$$

则由定义

$$\mathbb{E}[X] = \sum_x xp(x) = 1 \times 0.5 + (-1) \times 0.5 = 0$$

- 问题：实际中可能无法知道精确的分布！

▷ 方法一：基于模型

- 假设概率模型已知为

$$p(X = 1) = 0.5, \quad p(X = -1) = 0.5$$

则由定义

$$\mathbb{E}[X] = \sum_x xp(x) = 1 \times 0.5 + (-1) \times 0.5 = 0$$

- 问题：实际中可能无法知道精确的分布！

▷ 方法二：不依赖模型（无模型方法）

- 思路：多次抛硬币，然后计算结果的平均值。
- 假设我们得到样本序列 $\{x_1, x_2, \dots, x_N\}$ 。
则均值可近似为

$$\mathbb{E}[X] \approx \bar{x} = \frac{1}{N} \sum_{j=1}^N x_j.$$

这就是蒙特卡洛估计的思想！

▷ 方法二：不依赖模型（无模型方法）

- 思路：多次抛硬币，然后计算结果的平均值。
- 假设我们得到样本序列 $\{x_1, x_2, \dots, x_N\}$ 。
则均值可近似为

$$\mathbb{E}[X] \approx \bar{x} = \frac{1}{N} \sum_{j=1}^N x_j.$$

这就是蒙特卡洛估计的思想！

动机示例：蒙特卡洛估计

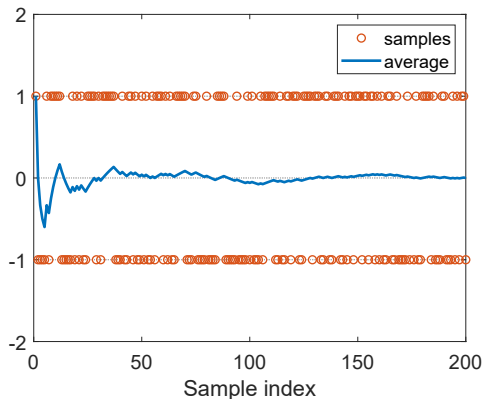
▷ **问题：**蒙特卡洛估计准确吗？

- 当 N 较小时，近似不准确。
- 当 N 较大时，近似更准确。

动机示例：蒙特卡洛估计

▷ 问题：蒙特卡洛估计准确吗？

- 当 N 较小时，近似不准确。
- 当 N 较大时，近似更准确。



Theorem (大数定律)

对于随机变量 X ，假设 $\{x_j\}_{j=1}^N$ 是一组独立同分布的样本。令 $\bar{x} = \frac{1}{N} \sum_{j=1}^N x_j$ 为样本均值。则

$$\begin{aligned}\mathbb{E}[\bar{x}] &= \mathbb{E}[X], \\ \text{Var}[\bar{x}] &= \frac{1}{N} \text{Var}[X].\end{aligned}$$

因此， $\bar{x}$ 是 $\mathbb{E}[X]$ 的无偏估计，且其方差随 N 趋于无穷而趋于零。

- ▷ 样本必须是独立同分布 (iid) 的
- ▷ 证明详见教材。

▷ 小结:

- 蒙特卡洛估计是指一大类依赖重复随机采样来求解近似问题的技术。
- 为什么关注蒙特卡洛估计？因为它不需要模型！
- 为什么关注均值估计？因为状态价值和动作价值均定义为随机变量的期望！

▷ 小结：

- 蒙特卡洛估计是指一大类依赖重复随机采样来求解近似问题的技术。
- 为什么关注蒙特卡洛估计？因为它不需要模型！
- 为什么关注均值估计？因为状态价值和动作价值均定义为随机变量的期望！

▷ 小结：

- 蒙特卡洛估计是指一大类依赖重复随机采样来求解近似问题的技术。
- 为什么关注蒙特卡洛估计？因为它不需要模型！
- 为什么关注均值估计？因为状态价值和动作价值均定义为随机变量的期望！

- 1 动机示例
- 2 最简单的基于蒙特卡洛的强化学习算法
 - 算法: MC Basic
- 3 更高效地利用数据
 - 算法: MC Exploring Starts
- 4 无需探索性起始的蒙特卡洛方法
 - 算法: MC ϵ -贪婪

将策略迭代转化为无模型算法

核心思想是将策略迭代算法转化为无模型算法。

若能

- 深入理解策略迭代算法，以及
 - 理解蒙特卡洛均值估计的思想，
- 则该转化过程便不难理解。

将策略迭代转化为无模型算法

策略迭代每次迭代包含两个步骤:

$$\begin{cases} \text{策略评估: } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{策略改进: } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

策略改进步骤的逐元素形式为:

$$\begin{aligned} \pi_{k+1}(s) &= \arg \max_{\pi} \sum_a \pi(a|s) \left[\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right] \\ &= \arg \max_{\pi} \sum_a \pi(a|s) q_{\pi_k}(s, a), \quad s \in \mathcal{S} \end{aligned}$$

关键在于计算 $q_{\pi_k}(s, a)$!

将策略迭代转化为无模型算法

策略迭代每次迭代包含两个步骤:

$$\begin{cases} \text{策略评估: } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{策略改进: } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

策略改进步骤的逐元素形式为:

$$\begin{aligned} \pi_{k+1}(s) &= \arg \max_{\pi} \sum_a \pi(a|s) \left[\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right] \\ &= \arg \max_{\pi} \sum_a \pi(a|s) q_{\pi_k}(s, a), \quad s \in \mathcal{S} \end{aligned}$$

关键在于计算 $q_{\pi_k}(s, a)$!

将策略迭代转化为无模型算法

策略迭代每次迭代包含两个步骤:

$$\begin{cases} \text{策略评估: } v_{\pi_k} = r_{\pi_k} + \gamma P_{\pi_k} v_{\pi_k} \\ \text{策略改进: } \pi_{k+1} = \arg \max_{\pi} (r_{\pi} + \gamma P_{\pi} v_{\pi_k}) \end{cases}$$

策略改进步骤的逐元素形式为:

$$\begin{aligned} \pi_{k+1}(s) &= \arg \max_{\pi} \sum_a \pi(a|s) \left[\sum_r p(r|s, a) r + \gamma \sum_{s'} p(s'|s, a) v_{\pi_k}(s') \right] \\ &= \arg \max_{\pi} \sum_a \pi(a|s) q_{\pi_k}(s, a), \quad s \in \mathcal{S} \end{aligned}$$

关键在于计算 $q_{\pi_k}(s, a)$!

将策略迭代转化为无模型算法

动作价值的两种表达形式:

- 形式一: 需要模型

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

- 形式二: 不需要模型

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

实现无模型强化学习的思路: 我们可以利用形式二, 基于数据 (样本或经验) 来获得 $q_{\pi_k}(s, a)$!

将策略迭代转化为无模型算法

动作价值的两种表达形式:

- 形式一: 需要模型

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

- 形式二: 不需要模型

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

实现无模型强化学习的思路: 我们可以利用形式二, 基于数据 (样本或经验) 来获得 $q_{\pi_k}(s, a)$!

将策略迭代转化为无模型算法

动作价值的两种表达形式:

- 形式一: 需要模型

$$q_{\pi_k}(s, a) = \sum_r p(r|s, a)r + \gamma \sum_{s'} p(s'|s, a)v_{\pi_k}(s')$$

- 形式二: 不需要模型

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

实现无模型强化学习的思路: 我们可以利用形式二, 基于数据 (样本或经验) 来获得 $q_{\pi_k}(s, a)$!

将策略迭代转化为无模型算法

如何基于数据获得 $q_{\pi_k}(s, a)$?

- 从 (s, a) 出发, 按照策略 π_k 生成一条轨迹 (episode)。
- 该轨迹的回报为 $g(s, a)$, 是

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- 假设我们有一个值轨迹, 从而得到 $\{g^{(j)}(s, a)\}$ 。则

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a).$$

根本思想: 当模型不可用时, 我们可以用数据代替。

将策略迭代转化为无模型算法

如何基于数据获得 $q_{\pi_k}(s, a)$?

- 从 (s, a) 出发，按照策略 π_k 生成一条轨迹 (episode)。
- 该轨迹的回报为 $g(s, a)$ ，是

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- 中 G_t 的一个样本。如果我们有多条轨迹，从而得到 $\{g^{(j)}(s, a)\}$ 。则

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a).$$

根本思想：当模型不可用时，我们可以用数据代替。

将策略迭代转化为无模型算法

如何基于数据获得 $q_{\pi_k}(s, a)$?

- 从 (s, a) 出发，按照策略 π_k 生成一条轨迹 (episode)。
- 该轨迹的回报为 $g(s, a)$ ，是

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- 假设我们有 N 条轨迹，从而得到 $\{g^{(j)}(s, a)\}$ 。则

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a).$$

根本思想：当模型不可用时，我们可以用数据代替。

将策略迭代转化为无模型算法

如何基于数据获得 $q_{\pi_k}(s, a)$?

- 从 (s, a) 出发，按照策略 π_k 生成一条轨迹 (episode)。
- 该轨迹的回报为 $g(s, a)$ ，是

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a]$$

- 假设我们有 N 个独立同分布的轨迹，从而得到 $\{g^{(j)}(s, a)\}$ 。则

$$q_{\pi_k}(s, a) = \mathbb{E}[G_t | S_t = s, A_t = a] \approx \frac{1}{N} \sum_{i=1}^N g^{(i)}(s, a).$$

根本思想：当模型不可用时，我们可以用数据代替。

给定初始策略 π_0 ，第 k 次迭代包含两个步骤。

- **步骤一：策略评估。** 该步骤的目标是对所有 (s, a) 估计 $q_{\pi_k}(s, a)$ 。具体地，对每个 (s, a) ，运行足够多条轨迹，将其回报的平均值记为 $q_k(s, a)$ ，用于近似 $q_{\pi_k}(s, a)$ 。
 - 策略迭代算法的第一步通过求解贝尔曼方程得到状态价值 v_{π_k} ，进而计算 $q_{\pi_k}(s, a)$ ，这需要模型。
 - *MC Basic* 算法的第一步直接从经验样本中估计 $q_k(s, a)$ ，不需要模型。
- **步骤二：策略改进。** 该步骤对所有 $s \in \mathcal{S}$ 求解
$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) q_k(s, a)。$$
贪婪最优策略为 $\pi_{k+1}(a_k^*|s) = 1$ ，其中 $a_k^* = \arg \max_a q_k(s, a)$ 。
 - 该步骤与策略迭代算法的第二步完全相同。

给定初始策略 π_0 ，第 k 次迭代包含两个步骤。

- **步骤一：策略评估。** 该步骤的目标是对所有 (s, a) 估计 $q_{\pi_k}(s, a)$ 。具体地，对每个 (s, a) ，运行足够多条轨迹，将其回报的平均值记为 $q_k(s, a)$ ，用于近似 $q_{\pi_k}(s, a)$ 。
 - 策略迭代算法的第一步通过求解贝尔曼方程得到状态价值 v_{π_k} ，进而计算 $q_{\pi_k}(s, a)$ ，这需要模型。
 - **MC Basic** 算法的第一步直接从经验样本中估计 $q_k(s, a)$ ，不需要模型。
- **步骤二：策略改进。** 该步骤对所有 $s \in \mathcal{S}$ 求解
$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) q_k(s, a)。$$
贪婪最优策略为 $\pi_{k+1}(a_k^*|s) = 1$ ，其中 $a_k^* = \arg \max_a q_k(s, a)$ 。
 - 该步骤与策略迭代算法的第二步完全相同。

给定初始策略 π_0 ，第 k 次迭代包含两个步骤。

- **步骤一：策略评估。** 该步骤的目标是对所有 (s, a) 估计 $q_{\pi_k}(s, a)$ 。具体地，对每个 (s, a) ，运行足够多条轨迹，将其回报的平均值记为 $q_k(s, a)$ ，用于近似 $q_{\pi_k}(s, a)$ 。
 - 策略迭代算法的第一步通过求解贝尔曼方程得到状态价值 v_{π_k} ，进而计算 $q_{\pi_k}(s, a)$ ，这需要模型。
 - **MC Basic** 算法的第一步直接从经验样本中估计 $q_k(s, a)$ ，不需要模型。
- **步骤二：策略改进。** 该步骤对所有 $s \in \mathcal{S}$ 求解
$$\pi_{k+1}(s) = \arg \max_{\pi} \sum_a \pi(a|s) q_k(s, a)。$$
贪婪最优策略为 $\pi_{k+1}(a_k^*|s) = 1$ ，其中 $a_k^* = \arg \max_a q_k(s, a)$ 。
 - 该步骤与策略迭代算法的第二步完全相同。

▷ 算法描述:

伪代码: **MC Basic 算法** (策略迭代的无模型变体)

初始化: 初始策略 π_0 。

目标: 搜索最优策略。

第 k 次迭代 ($k = 0, 1, 2, \dots$), 执行:

对每个状态 $s \in \mathcal{S}$, 执行:

对每个动作 $a \in \mathcal{A}(s)$, 执行:

从 (s, a) 出发, 按照 π_k 收集足够多条轨迹

策略评估:

$q_{\pi_k}(s, a) \approx q_k(s, a) =$ 所有从 (s, a) 出发的轨迹的平均回报

策略改进:

$a_k^*(s) = \arg \max_a q_k(s, a)$

若 $a = a_k^*$ 则 $\pi_{k+1}(a|s) = 1$, 否则 $\pi_{k+1}(a|s) = 0$

- MC Basic 是策略迭代算法的一个变体。
- 无模型算法建立在有模型算法的基础之上。因此，在学习无模型算法之前，有必要先深入理解有模型算法。
- MC Basic 有助于揭示基于蒙特卡洛的无模型强化学习的核心思想，但由于效率较低，在实际中并不实用。
- 为什么 MC Basic 估计的是动作价值而非状态价值？因为在没有模型的情况下，状态价值无法直接用于改进策略，应当直接估计动作价值。
- 由于策略迭代算法是收敛的，在轨迹数量足够的情况下，MC Basic 的收敛性同样得到保证。

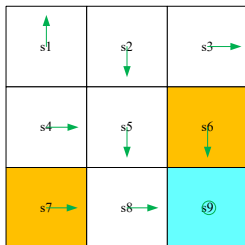
- MC Basic 是策略迭代算法的一个变体。
- 无模型算法建立在有模型算法的基础之上。因此，在学习无模型算法之前，有必要先深入理解有模型算法。
- MC Basic 有助于揭示基于蒙特卡洛的无模型强化学习的核心思想，但由于效率较低，在实际中并不实用。
- 为什么 MC Basic 估计的是动作价值而非状态价值？因为在没有模型的情况下，状态价值无法直接用于改进策略，应当直接估计动作价值。
- 由于策略迭代算法是收敛的，在轨迹数量足够的情况下，MC Basic 的收敛性同样得到保证。

- MC Basic 是策略迭代算法的一个变体。
- 无模型算法建立在有模型算法的基础之上。因此，在学习无模型算法之前，有必要先深入理解有模型算法。
- MC Basic 有助于揭示基于蒙特卡洛的无模型强化学习的核心思想，但由于效率较低，在实际中并不实用。
- 为什么 MC Basic 估计的是动作价值而非状态价值？因为在没有模型的情况下，状态价值无法直接用于改进策略，应当直接估计动作价值。
- 由于策略迭代算法是收敛的，在轨迹数量足够的情况下，MC Basic 的收敛性同样得到保证。

- MC Basic 是策略迭代算法的一个变体。
- 无模型算法建立在有模型算法的基础之上。因此，在学习无模型算法之前，有必要先深入理解有模型算法。
- MC Basic 有助于揭示基于蒙特卡洛的无模型强化学习的核心思想，但由于效率较低，在实际中并不实用。
- 为什么 MC Basic 估计的是动作价值而非状态价值？因为在没有模型的情况下，状态价值无法直接用于改进策略，应当直接估计动作价值。
- 由于策略迭代算法是收敛的，在轨迹数量足够的情况下，MC Basic 的收敛性同样得到保证。

- MC Basic 是策略迭代算法的一个变体。
- 无模型算法建立在有模型算法的基础之上。因此，在学习无模型算法之前，有必要先深入理解有模型算法。
- MC Basic 有助于揭示基于蒙特卡洛的无模型强化学习的核心思想，但由于效率较低，在实际中并不实用。
- 为什么 MC Basic 估计的是动作价值而非状态价值？因为在没有模型的情况下，状态价值无法直接用于改进策略，应当直接估计动作价值。
- 由于策略迭代算法是收敛的，在轨迹数量足够的情况下，MC Basic 的收敛性同样得到保证。

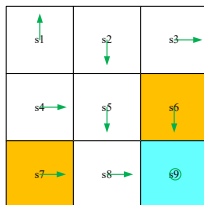
示例一：逐步演示



任务：

- 初始策略如图所示。
- 使用 MC Basic 寻找最优策略。
- $r_{\text{boundary}} = -1$, $r_{\text{forbidden}} = -1$, $r_{\text{target}} = 1$, $\gamma = 0.9$ 。

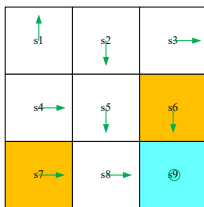
示例一：逐步演示



流程：给定当前策略 π_k

- 步骤一——策略评估：计算 $q_{\pi_k}(s, a)$
共有多少状态-动作对？ 9个状态 $\times$ 5个动作 = 45个状态-动作对！
- 步骤二——策略改进：选择贪婪动作 $a^*(s) = \arg \max_a q_{\pi_k}(s, a)$

示例一：逐步演示

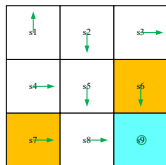


▷ 由于篇幅限制，仅展示 $q_{\pi_k}(s_1, a)$

▷ 步骤一——策略评估：

- 由于当前策略是确定性的，一条轨迹就足以得到动作价值！否则，若策略或模型是随机的，则需要许多条轨迹！

示例一：逐步演示



- 从 (s_1, a_1) 出发, 轨迹为 $s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_1) = -1 + \gamma(-1) + \gamma^2(-1) + \dots$$

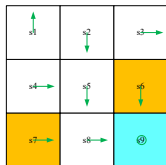
- 从 (s_1, a_2) 出发, 轨迹为 $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_2) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3(1) + \gamma^4(1) + \dots$$

- 从 (s_1, a_3) 出发, 轨迹为 $s_1 \xrightarrow{a_3} s_4 \xrightarrow{a_2} s_5 \xrightarrow{a_3} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_3) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3(1) + \gamma^4(1) + \dots$$

示例一：逐步演示



- 从 (s_1, a_1) 出发, 轨迹为 $s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_1) = -1 + \gamma(-1) + \gamma^2(-1) + \dots$$

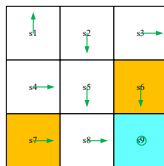
- 从 (s_1, a_2) 出发, 轨迹为 $s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_3} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_2) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3(1) + \gamma^4(1) + \dots$$

- 从 (s_1, a_3) 出发, 轨迹为 $s_1 \xrightarrow{a_3} s_4 \xrightarrow{a_2} s_5 \xrightarrow{a_3} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_3) = 0 + \gamma 0 + \gamma^2 0 + \gamma^3(1) + \gamma^4(1) + \dots$$

示例一：逐步演示



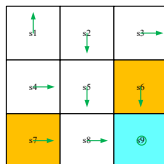
- 从 (s_1, a_4) 出发，轨迹为 $s_1 \xrightarrow{a_4} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_4) = -1 + \gamma(-1) + \gamma^2(-1) + \dots$$

- 从 (s_1, a_5) 出发，轨迹为 $s_1 \xrightarrow{a_5} s_1 \xrightarrow{a_1} s_1 \xrightarrow{a_1} \dots$ 。故动作价值为

$$q_{\pi_0}(s_1, a_5) = 0 + \gamma(-1) + \gamma^2(-1) + \dots$$

示例一：逐步演示



▷ 步骤二——策略改进:

- 观察动作价值，可以发现

$$q_{\pi_0}(s_1, a_2) = q_{\pi_0}(s_1, a_3)$$

取得最大值。

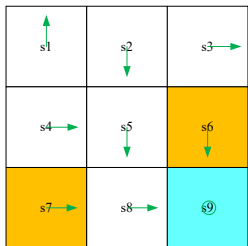
- 因此，策略可改进为

$$\pi_1(a_2|s_1) = 1 \quad \text{或} \quad \pi_1(a_3|s_1) = 1$$

无论哪种选择， s_1 的新策略都达到最优。

对此简单示例，仅需一次迭代即可！

示例一：逐步演示



练习：现在使用 MC Basic 更新 s_3 的策略！

- 1 动机示例
- 2 最简单的基于蒙特卡洛的强化学习算法
 - 算法: MC Basic
- 3 更高效地利用数据
 - 算法: MC Exploring Starts
- 4 无需探索性起始的蒙特卡洛方法
 - 算法: MC ϵ -贪婪

MC Basic 算法:

- 优点: 核心思想清晰易懂!
- 缺点: 过于简单, 缺乏实用性。

然而, MC Basic 可以被扩展得更加高效。

▷ 考虑一个网格世界示例，按照策略 π 生成如下轨迹：

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$$

▷ 访问 (**Visit**)：每当一个状态-动作对出现在轨迹中，称为对该状态-动作对的一次访问。

▷ 利用数据的方法：初始访问法 (**Initial-visit method**)

- 仅计算并近似 $q_{\pi}(s_1, a_2)$ 的回报。
- 这正是 MC Basic 算法所采用的方式。
- 缺点：未能充分利用数据。

▷ 考虑一个网格世界示例，按照策略 π 生成如下轨迹：

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$$

▷ **访问 (Visit)**：每当一个状态-动作对出现在轨迹中，称为对该状态-动作对的一次访问。

▷ 利用数据的方法：初始访问法 (Initial-visit method)

- 仅计算并近似 $q_\pi(s_1, a_2)$ 的回报。
- 这正是 MC Basic 算法所采用的方式。
- 缺点：未能充分利用数据。

▷ 考虑一个网格世界示例，按照策略 π 生成如下轨迹：

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots$$

▷ **访问 (Visit)**：每当一个状态-动作对出现在轨迹中，称为对该状态-动作对的一次访问。

▷ 利用数据的方法：**初始访问法 (Initial-visit method)**

- 仅计算并近似 $q_\pi(s_1, a_2)$ 的回报。
- 这正是 MC Basic 算法所采用的方式。
- 缺点：未能充分利用数据。

▷ 该轨迹还访问了其他状态-动作对。

$$s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots \quad [\text{原始轨迹}]$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$ 、.....

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

▷ 该轨迹还访问了其他状态-动作对。

$$\begin{aligned}s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[原始轨迹]} \\s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_4) \text{ 出发的轨迹]}\end{aligned}$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$ 、.....

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

▷ 该轨迹还访问了其他状态-动作对。

$$\begin{aligned}s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[原始轨迹]} \\s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_4) \text{ 出发的轨迹]} \\s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_1, a_2) \text{ 出发的轨迹]}\end{aligned}$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$ 、.....

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

▷ 该轨迹还访问了其他状态-动作对。

$$\begin{aligned}s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[原始轨迹]} \\s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_4) \text{ 出发的轨迹]} \\s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_1, a_2) \text{ 出发的轨迹]} \\s_2 &\xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_3) \text{ 出发的轨迹]}\end{aligned}$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$ 、.....

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

▷ 该轨迹还访问了其他状态-动作对。

$$\begin{aligned}s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[原始轨迹]} \\s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_4) \text{ 出发的轨迹]} \\s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_1, a_2) \text{ 出发的轨迹]} \\s_2 &\xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_3) \text{ 出发的轨迹]} \\s_5 &\xrightarrow{a_1} \dots && \text{[从 } (s_5, a_1) \text{ 出发的轨迹]}\end{aligned}$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$ 、.....

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

▷ 该轨迹还访问了其他状态-动作对。

$$\begin{aligned}s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[原始轨迹]} \\s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_4) \text{ 出发的轨迹]} \\s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_1, a_2) \text{ 出发的轨迹]} \\s_2 &\xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_3) \text{ 出发的轨迹]} \\s_5 &\xrightarrow{a_1} \dots && \text{[从 } (s_5, a_1) \text{ 出发的轨迹]}\end{aligned}$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

▷ 该轨迹还访问了其他状态-动作对。

$$\begin{aligned}s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[原始轨迹]} \\s_2 &\xrightarrow{a_4} s_1 \xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_4) \text{ 出发的轨迹]} \\s_1 &\xrightarrow{a_2} s_2 \xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_1, a_2) \text{ 出发的轨迹]} \\s_2 &\xrightarrow{a_3} s_5 \xrightarrow{a_1} \dots && \text{[从 } (s_2, a_3) \text{ 出发的轨迹]} \\s_5 &\xrightarrow{a_1} \dots && \text{[从 } (s_5, a_1) \text{ 出发的轨迹]}\end{aligned}$$

可用于估计 $q_\pi(s_1, a_2)$ 、 $q_\pi(s_2, a_4)$ 、 $q_\pi(s_2, a_3)$ 、 $q_\pi(s_5, a_1)$

高效利用数据的方法:

- 首次访问法 (first-visit method)
- 每次访问法 (every-visit method)

- ▷ 基于蒙特卡洛的强化学习中，另一个方面是[何时更新策略](#)。有两种方法。
 - 方法一：在策略评估步骤中，收集从某一状态-动作对出发的所有轨迹，再用平均回报来近似动作价值。
 - 这是 MC Basic 算法所采用的方式。
 - 该方法的问题在于，智能体必须等待所有轨迹收集完毕。
 - 方法二：用单条轨迹的回报来近似动作价值。
 - 这样可以逐条轨迹地改进策略。

- ▷ 基于蒙特卡洛的强化学习中，另一个方面是**何时更新策略**。有两种方法。
 - **方法一**：在策略评估步骤中，收集从某一状态-动作对出发的**所有**轨迹，再用平均回报来近似动作价值。
 - 这是 **MC Basic** 算法所采用的方式。
 - 该方法的问题在于，智能体必须**等待**所有轨迹收集完毕。
 - **方法二**：用单条轨迹的回报来近似动作价值。
 - 这样可以逐条轨迹地改进策略。

- ▷ 基于蒙特卡洛的强化学习中，另一个方面是**何时更新策略**。有两种方法。
 - **方法一**：在策略评估步骤中，收集从某一状态-动作对出发的**所有**轨迹，再用平均回报来近似动作价值。
 - 这是 **MC Basic** 算法所采用的方式。
 - 该方法的问题在于，智能体必须**等待**所有轨迹收集完毕。
 - **方法二**：用**单条**轨迹的回报来近似动作价值。
 - 这样可以**逐条轨迹**地改进策略。

▷ 方法二会带来问题吗？

- 有人可能认为，单条轨迹的回报不能准确近似对应的动作价值。
- 实际上，我们在上一章介绍的截断策略迭代算法中已经这样做过了！

▷ 广义策略迭代（Generalized Policy Iteration）：

- 并非一个具体算法。
- 指在策略评估与策略改进过程之间交替进行的通用思想或框架。
- 许多强化学习算法都属于这一框架。

▷ 方法二会带来问题吗？

- 有人可能认为，单条轨迹的回报不能准确近似对应的动作价值。
- 实际上，我们在上一章介绍的截断策略迭代算法中已经这样做过了！

▷ 广义策略迭代（Generalized Policy Iteration）：

- 并非一个具体算法。
- 指在策略评估与策略改进过程之间交替进行的通用思想或框架。
- 许多强化学习算法都属于这一框架。

MC Exploring Starts (探索性起始)

▷ 若我们更高效地利用数据并更新估计，可得到一种新算法——MC Exploring Starts:

算法: MC Exploring Starts (MC Basic 的高效变体)

初始化: 初始策略 $\pi_0(a|s)$ 和所有 (s, a) 的初始价值 $q(s, a)$ 。对所有 (s, a) ，令 $\text{Returns}(s, a) = 0$ ， $\text{Num}(s, a) = 0$ 。

目标: 搜索最优策略。

对每条轨迹，执行:

生成轨迹: 选择起始状态-动作对 (s_0, a_0) ，并确保所有状态-动作对均有可能被选中（即探索性起始条件）。按当前策略生成长度为 T 的轨迹: $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 。

每条轨迹的初始化: $g \leftarrow 0$

对轨迹的每个时间步 $t = T - 1, T - 2, \dots, 0$ ，执行:

$$g \leftarrow \gamma g + r_{t+1}$$

$$\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$$

$$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$$

策略评估:

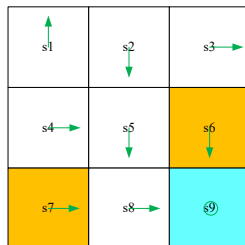
$$q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$$

策略改进:

若 $a = \arg \max_a q(s_t, a)$ 则 $\pi(a|s_t) = 1$ ，否则 $\pi(a|s_t) = 0$

▷ 什么是探索性起始 (Exploring Starts) ?

- 探索性起始是指需要生成足够多的轨迹，使每个状态-动作对都作为起始点被覆盖到。
- 例如，需要从 $\{(s_1, a_j)\}_{j=1}^5$ 、 $\{(s_2, a_j)\}_{j=1}^5$ 、 $\dots$ 、 $\{(s_9, a_j)\}_{j=1}^5$ 各自出发的轨迹。
否则，若没有从 (s_i, a_j) 出发的轨迹，则 (s_i, a_j) 可能不会被任何轨迹访问到，从而无法估计 $q_\pi(s_i, a_j)$ 。
- MC Basic 和 MC Exploring Starts 均需要该假设。



▷ 为什么需要探索性起始？

- 理论上，只有每个状态的每个动作价值都被充分探索，才能正确选择最优动作。

否则，若某个动作从未被探索过，而它恰好是最优动作，则会被遗漏。

- 实践中，探索性起始难以实现。对于许多应用（尤其是与环境发生物理交互的场景），很难收集从每个状态-动作对出发的轨迹。

能否去掉探索性起始的要求？下面将说明，通过使用软策略可以做到这一点。

▷ 为什么需要探索性起始？

- 理论上，只有每个状态的每个动作价值都被充分探索，才能正确选择最优动作。

否则，若某个动作从未被探索过，而它恰好是最优动作，则会被遗漏。

- 实践中，探索性起始难以实现。对于许多应用（尤其是与环境发生物理交互的场景），很难收集从每个状态-动作对出发的轨迹。

能否去掉探索性起始的要求？下面将说明，通过使用软策略可以做到这一点。

▷ 为什么需要探索性起始？

- 理论上，只有每个状态的每个动作价值都被充分探索，才能正确选择最优动作。

否则，若某个动作从未被探索过，而它恰好是最优动作，则会被遗漏。

- 实践中，探索性起始难以实现。对于许多应用（尤其是与环境发生物理交互的场景），很难收集从每个状态-动作对出发的轨迹。

能否去掉探索性起始的要求？下面将说明，通过使用软策略可以做到这一点。

- 1 动机示例
- 2 最简单的基于蒙特卡洛的强化学习算法
 - 算法: MC Basic
- 3 更高效地利用数据
 - 算法: MC Exploring Starts
- 4 无需探索性起始的蒙特卡洛方法
 - 算法: MC ϵ -贪婪

▷ 什么是软策略 (Soft Policy) ?

- 若采取任何动作的概率均为正，则该策略为软策略。
 - 确定性策略：例如，贪婪策略
 - 随机策略：例如，软策略

▷ 为什么引入软策略？

- 采用软策略时，若轨迹足够长，少量轨迹便可访问每个状态-动作对。
- 因此，无需从每个状态-动作对出发收集大量轨迹，从而可以去掉探索性起始的要求。

▷ 什么是软策略 (Soft Policy) ?

- 若采取任何动作的概率均为正，则该策略为软策略。
 - 确定性策略：例如，贪婪策略
 - 随机策略：例如，软策略

▷ 为什么引入软策略？

- 采用软策略时，若轨迹足够长，少量轨迹便可访问每个状态-动作对。
- 因此，无需从每个状态-动作对出发收集大量轨迹，从而可以去掉探索性起始的要求。

▷ 什么是软策略 (Soft Policy) ?

- 若采取任何动作的概率均为正，则该策略为软策略。
 - 确定性策略：例如，贪婪策略
 - 随机策略：例如，软策略

▷ 为什么引入软策略?

- 采用软策略时，若轨迹足够长，少量轨迹便可访问每个状态-动作对。
- 因此，无需从每个状态-动作对出发收集大量轨迹，从而可以去掉探索性起始的要求。

- 我们将使用哪种软策略？答： ϵ -贪婪策略 (ϵ -greedy policy)
- 什么是 ϵ -贪婪策略？

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

其中 $\epsilon \in [0, 1]$, $|\mathcal{A}(s)|$ 是状态 s 的动作数量。

- 示例：若 $\epsilon = 0.2$ ，则

$$\frac{\epsilon}{|\mathcal{A}(s)|} = \frac{0.2}{5} = 0.04, \quad 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - 0.04 \times 4 = 0.84$$

- 选择贪婪动作的概率始终大于其他动作，因为

$$1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

- 我们将使用哪种软策略？答： ϵ -贪婪策略 (ϵ -greedy policy)
- 什么是 ϵ -贪婪策略？

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

其中 $\epsilon \in [0, 1]$, $|\mathcal{A}(s)|$ 是状态 s 的动作数量。

- 示例：若 $\epsilon = 0.2$, 则

$$\frac{\epsilon}{|\mathcal{A}(s)|} = \frac{0.2}{5} = 0.04, \quad 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - 0.04 \times 4 = 0.84$$

- 选择贪婪动作的概率始终大于其他动作，因为

$$1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

- 我们将使用哪种软策略？答： ϵ -贪婪策略 (ϵ -greedy policy)
- 什么是 ϵ -贪婪策略？

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

其中 $\epsilon \in [0, 1]$, $|\mathcal{A}(s)|$ 是状态 s 的动作数量。

- 示例：若 $\epsilon = 0.2$ ，则

$$\frac{\epsilon}{|\mathcal{A}(s)|} = \frac{0.2}{5} = 0.04, \quad 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - 0.04 \times 4 = 0.84$$

- 选择贪婪动作的概率始终大于其他动作，因为

$$1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

- 我们将使用哪种软策略？答： ϵ -贪婪策略 (ϵ -greedy policy)
- 什么是 ϵ -贪婪策略？

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1), & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

其中 $\epsilon \in [0, 1]$, $|\mathcal{A}(s)|$ 是状态 s 的动作数量。

- 示例：若 $\epsilon = 0.2$ ，则

$$\frac{\epsilon}{|\mathcal{A}(s)|} = \frac{0.2}{5} = 0.04, \quad 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - 0.04 \times 4 = 0.84$$

- 选择贪婪动作的概率始终大于其他动作，因为

$$1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1 - \epsilon + \frac{\epsilon}{|\mathcal{A}(s)|} \geq \frac{\epsilon}{|\mathcal{A}(s)|}$$

ϵ -贪婪策略能够平衡利用与探索。

- 当 $\epsilon \rightarrow 0$ 时，退化为贪婪策略！

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = 0, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多利用，更少探索。

- 当 $\epsilon \rightarrow 1$ 时，退化为均匀分布。

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = \frac{1}{|\mathcal{A}(s)|}, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = \frac{1}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多探索，更少利用。

ϵ -贪婪策略能够平衡利用与探索。

- 当 $\epsilon \rightarrow 0$ 时，退化为贪婪策略！

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = 0, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多利用，更少探索。

- 当 $\epsilon \rightarrow 1$ 时，退化为均匀分布。

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = \frac{1}{|\mathcal{A}(s)|}, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = \frac{1}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多探索，更少利用。

ϵ -贪婪策略能够平衡利用与探索。

- 当 $\epsilon \rightarrow 0$ 时，退化为贪婪策略！

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = 0, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多利用，更少探索。

- 当 $\epsilon \rightarrow 1$ 时，退化为均匀分布。

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = \frac{1}{|\mathcal{A}(s)|}, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = \frac{1}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多探索，更少利用。

ϵ -贪婪策略能够平衡利用与探索。

- 当 $\epsilon \rightarrow 0$ 时，退化为贪婪策略！

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = 0, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多利用，更少探索。

- 当 $\epsilon \rightarrow 1$ 时，退化为均匀分布。

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = \frac{1}{|\mathcal{A}(s)|}, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = \frac{1}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多探索，更少利用。

ϵ -贪婪策略能够平衡利用与探索。

- 当 $\epsilon \rightarrow 0$ 时，退化为贪婪策略！

$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = 1, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = 0, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多利用，更少探索。

- 当 $\epsilon \rightarrow 1$ 时，退化为均匀分布。

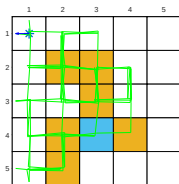
$$\pi(a|s) = \begin{cases} 1 - \frac{\epsilon}{|\mathcal{A}(s)|} (|\mathcal{A}(s)| - 1) = \frac{1}{|\mathcal{A}(s)|}, & \text{对贪婪动作,} \\ \frac{\epsilon}{|\mathcal{A}(s)|} = \frac{1}{|\mathcal{A}(s)|}, & \text{对其余 } |\mathcal{A}(s)| - 1 \text{ 个动作。} \end{cases}$$

更多探索，更少利用。

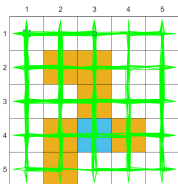
探索能力演示示例

▷ 单条轨迹能否访问所有状态-动作对？

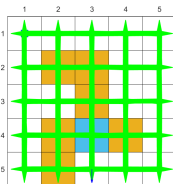
当 $\epsilon = 1$ 时，策略（均匀分布）具有最强的探索能力。



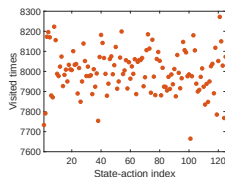
(a) 100步



(b) 1000步



(c) 10000步



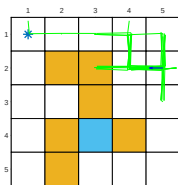
(d)

[点击此处播放视频](#) (视频仅在本机上)

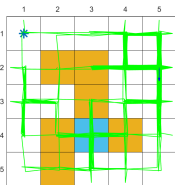
探索能力演示示例

▷ 单条轨迹能否访问所有状态-动作对？

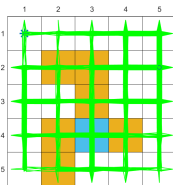
当 ϵ 较小时，策略的探索能力也较弱。



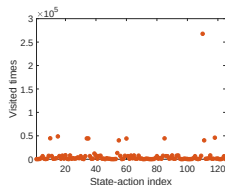
(a) 100步



(b) 1000步



(c) 10000步



(d)

▷ 如何将 ϵ -贪婪策略嵌入基于蒙特卡洛的强化学习算法？

原来，MC Basic 和 MC Exploring Starts 中策略改进步骤的目标是求解

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

其中 Π 表示所有可能策略的集合。此时的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*, \\ 0, & a \neq a_k^*, \end{cases}$$

其中 $a_k^* = \arg \max_a q_{\pi_k}(s, a)$ 。

▷ 如何将 ϵ -贪婪策略嵌入基于蒙特卡洛的强化学习算法？

原来，MC Basic 和 MC Exploring Starts 中策略改进步骤的目标是求解

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

其中 Π 表示所有可能策略的集合。此时的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*, \\ 0, & a \neq a_k^*, \end{cases}$$

其中 $a_k^* = \arg \max_a q_{\pi_k}(s, a)$ 。

▷ 如何将 ϵ -贪婪策略嵌入基于蒙特卡洛的强化学习算法？

原来，MC Basic 和 MC Exploring Starts 中策略改进步骤的目标是求解

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

其中 Π 表示所有可能策略的集合。此时的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1, & a = a_k^*, \\ 0, & a \neq a_k^*, \end{cases}$$

其中 $a_k^* = \arg \max_a q_{\pi_k}(s, a)$ 。

▷ 如何将 ϵ -贪婪策略嵌入基于蒙特卡洛的强化学习算法？

现在，策略改进步骤改为求解

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_\epsilon} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

其中 Π_ϵ 表示固定 ϵ 值的所有 ϵ -贪婪策略的集合。此时的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|} \epsilon, & a = a_k^*, \\ \frac{1}{|\mathcal{A}(s)|} \epsilon, & a \neq a_k^*. \end{cases}$$

小结：

- MC ϵ -贪婪与 MC Exploring Starts 相同，区别仅在于前者使用 ϵ -贪婪策略。
- MC ϵ -贪婪不要求探索性起始条件，但仍以另一种形式要求访问所有状态-动作对。

▷ 如何将 ϵ -贪婪策略嵌入基于蒙特卡洛的强化学习算法？

现在，策略改进步骤改为求解

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_\epsilon} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

其中 Π_ϵ 表示固定 ϵ 值的所有 ϵ -贪婪策略的集合。此时的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|}\epsilon, & a = a_k^*, \\ \frac{1}{|\mathcal{A}(s)|}\epsilon, & a \neq a_k^*. \end{cases}$$

小结：

- MC ϵ -贪婪与 MC Exploring Starts 相同，区别仅在于前者使用 ϵ -贪婪策略。
- MC ϵ -贪婪不要求探索性起始条件，但仍以另一种形式要求访问所有状态-动作对。

▷ 如何将 ϵ -贪婪策略嵌入基于蒙特卡洛的强化学习算法？

现在，策略改进步骤改为求解

$$\pi_{k+1}(s) = \arg \max_{\pi \in \Pi_{\epsilon}} \sum_a \pi(a|s) q_{\pi_k}(s, a),$$

其中 Π_{ϵ} 表示固定 ϵ 值的所有 ϵ -贪婪策略的集合。此时的最优策略为

$$\pi_{k+1}(a|s) = \begin{cases} 1 - \frac{|\mathcal{A}(s)|-1}{|\mathcal{A}(s)|} \epsilon, & a = a_k^*, \\ \frac{1}{|\mathcal{A}(s)|} \epsilon, & a \neq a_k^*. \end{cases}$$

小结：

- MC ϵ -贪婪与 MC Exploring Starts 相同，区别仅在于前者使用 ϵ -贪婪策略。
- MC ϵ -贪婪不要求探索性起始条件，但仍以另一种形式要求访问所有状态-动作对。

算法: MC ϵ -贪婪 (MC Exploring Starts 的变体)

初始化: 初始策略 $\pi_0(a|s)$ 和所有 (s, a) 的初始价值 $q(s, a)$ 。对所有 (s, a) , 令 $\text{Returns}(s, a) = 0$, $\text{Num}(s, a) = 0$ 。 $\epsilon \in (0, 1]$

目标: 搜索最优策略。

对每条轨迹, 执行:

生成轨迹: 选择起始状态-动作对 (s_0, a_0) (无需满足探索性起始条件)。按当前策略

生成长度为 T 的轨迹: $s_0, a_0, r_1, \dots, s_{T-1}, a_{T-1}, r_T$ 。

每条轨迹的初始化: $g \leftarrow 0$

对轨迹的每个时间步 $t = T - 1, T - 2, \dots, 0$, 执行:

$$g \leftarrow \gamma g + r_{t+1}$$

$$\text{Returns}(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) + g$$

$$\text{Num}(s_t, a_t) \leftarrow \text{Num}(s_t, a_t) + 1$$

策略评估:

$$q(s_t, a_t) \leftarrow \text{Returns}(s_t, a_t) / \text{Num}(s_t, a_t)$$

策略改进:

令 $a^* = \arg \max_a q(s_t, a)$, 且

$$\pi(a|s_t) = \begin{cases} 1 - \frac{|\mathcal{A}(s_t)|-1}{|\mathcal{A}(s_t)|} \epsilon, & a = a^* \\ \frac{1}{|\mathcal{A}(s_t)|} \epsilon, & a \neq a^* \end{cases}$$

▷ 演示 MC ϵ -贪婪算法。

每次迭代中，执行以下步骤：

- 在轨迹生成步骤中，使用前一策略生成一条长达100万步的轨迹！
- 在后续步骤中，用该单条轨迹更新策略。
- 两次迭代即可得到最优 ϵ -贪婪策略。

此处， $r_{\text{boundary}} = -1$ ， $r_{\text{forbidden}} = -10$ ， $r_{\text{target}} = 1$ ， $\gamma = 0.9$ 。

示例

▷ 演示 MC ϵ -贪婪算法。

每次迭代中，执行以下步骤：

- 在轨迹生成步骤中，使用前一策略生成一条长达100万步的轨迹！
- 在后续步骤中，用该单条轨迹更新策略。
- 两次迭代即可得到最优 ϵ -贪婪策略。

此处， $r_{\text{boundary}} = -1$ ， $r_{\text{forbidden}} = -10$ ， $r_{\text{target}} = 1$ ， $\gamma = 0.9$ 。

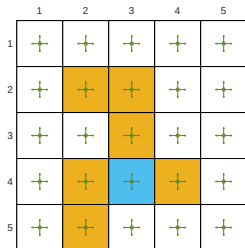
示例

▷ 演示 MC ϵ -贪婪算法。

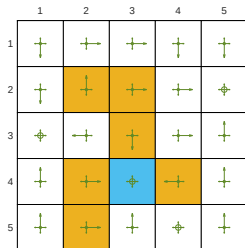
每次迭代中，执行以下步骤：

- 在轨迹生成步骤中，使用前一策略生成一条长达100万步的轨迹！
- 在后续步骤中，用该单条轨迹更新策略。
- 两次迭代即可得到最优 ϵ -贪婪策略。

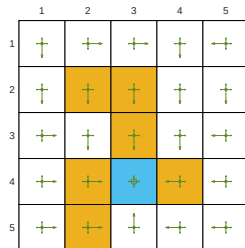
此处， $r_{\text{boundary}} = -1$ ， $r_{\text{forbidden}} = -10$ ， $r_{\text{target}} = 1$ ， $\gamma = 0.9$ 。



(a) 初始策略



(b) 第一次迭代后



(c) 第二次迭代后

- ▷ 与贪婪策略相比，
 - ϵ -贪婪策略的**优点**在于，当 ϵ 较大时具有较强的探索能力。
 - 因此无需满足探索性起始条件。
 - **缺点**是 ϵ -贪婪策略通常**并不最优**。
 - 它仅在所有 ϵ -贪婪策略的集合 Π_ϵ 中最优。
 - 因此 ϵ 不应过大，也可以使用衰减的 ϵ 。

▷ 接下来，通过示例进行演示。设置为

$$r_{\text{boundary}} = -1, r_{\text{forbidden}} = -10, r_{\text{target}} = 1, \gamma = 0.9$$

- ▷ 与贪婪策略相比，
 - ϵ -贪婪策略的**优点**在于，当 ϵ 较大时具有较强的探索能力。
 - 因此无需满足探索性起始条件。
 - **缺点**是 ϵ -贪婪策略通常**并不最优**。
 - 它仅在所有 ϵ -贪婪策略的集合 Π_ϵ 中最优。
 - 因此 ϵ 不应过大，也可以使用衰减的 ϵ 。

▷ 接下来，通过示例进行演示。设置为

$$r_{\text{boundary}} = -1, \quad r_{\text{forbidden}} = -10, \quad r_{\text{target}} = 1, \quad \gamma = 0.9$$

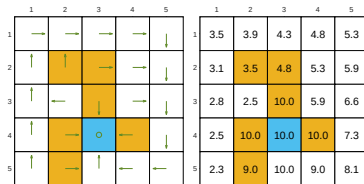
▷ 示例：给定 ϵ -贪婪策略，其状态价值是多少？



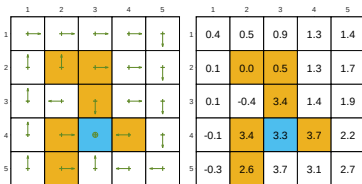
$$\epsilon = 0$$

- ▷ 当 ϵ 增大时，策略的最优性变差！
- ▷ 为什么目标状态的状态价值为负？

▷ 示例：给定 ϵ -贪婪策略，其状态价值是多少？



$\epsilon = 0$



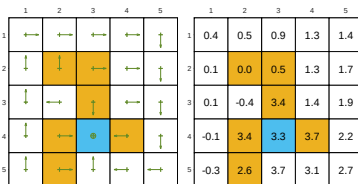
$\epsilon = 0.1$

- ▷ 当 ϵ 增大时，策略的最优性变差！
- ▷ 为什么目标状态的状态价值为负？

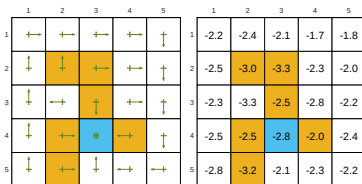
▷ 示例：给定 ϵ -贪婪策略，其状态价值是多少？



$\epsilon = 0$



$\epsilon = 0.1$

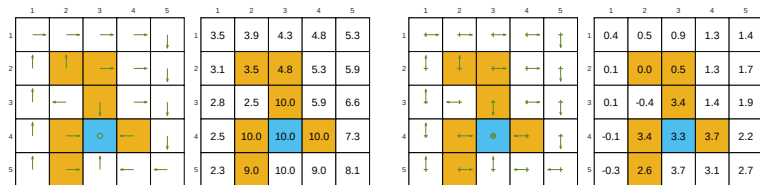


$\epsilon = 0.2$

▷ 当 ϵ 增大时，策略的最优性变差！

▷ 为什么目标状态的状态价值为负？

▷ 示例：给定 ϵ -贪婪策略，其状态价值是多少？



$\epsilon = 0$

$\epsilon = 0.1$



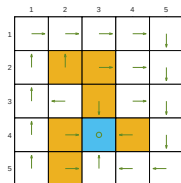
$\epsilon = 0.2$

$\epsilon = 0.5$

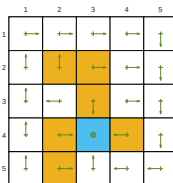
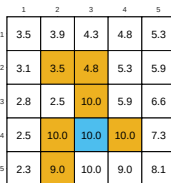
▷ 当 ϵ 增大时，策略的最优性变差！

▷ 为什么目标状态的状态价值为负？

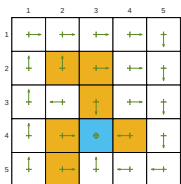
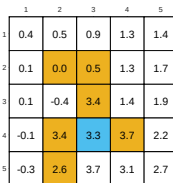
▷ 示例：给定 ϵ -贪婪策略，其状态价值是多少？



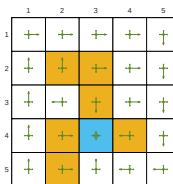
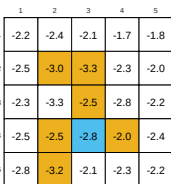
$\epsilon = 0$



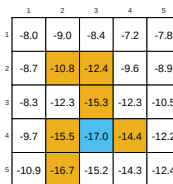
$\epsilon = 0.1$



$\epsilon = 0.2$



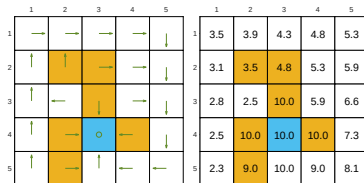
$\epsilon = 0.5$



▷ 当 ϵ 增大时，策略的最优性变差！

▷ 为什么目标状态的状态价值为负？

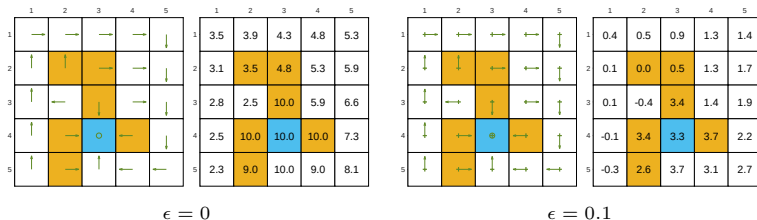
▷ 示例：找到最优 ϵ -贪婪策略及其状态价值。



$$\epsilon = 0$$

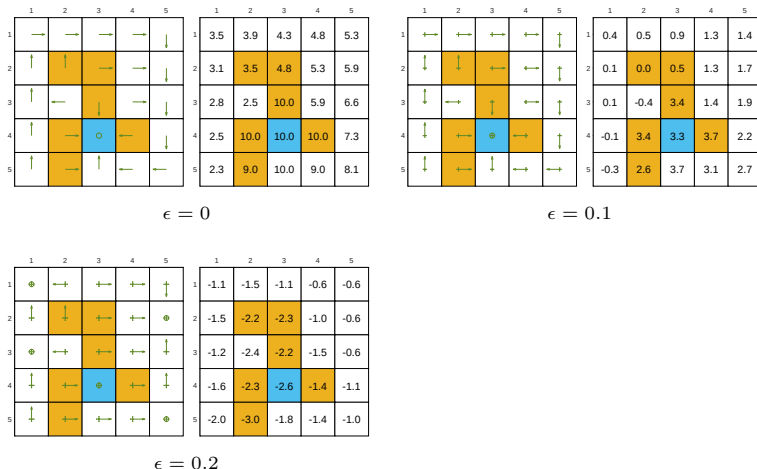
▷ 最优 ϵ -贪婪策略与贪婪最优策略并不一致！为什么？以目标状态为例思考。

▷ 示例：找到最优 ϵ -贪婪策略及其状态价值。



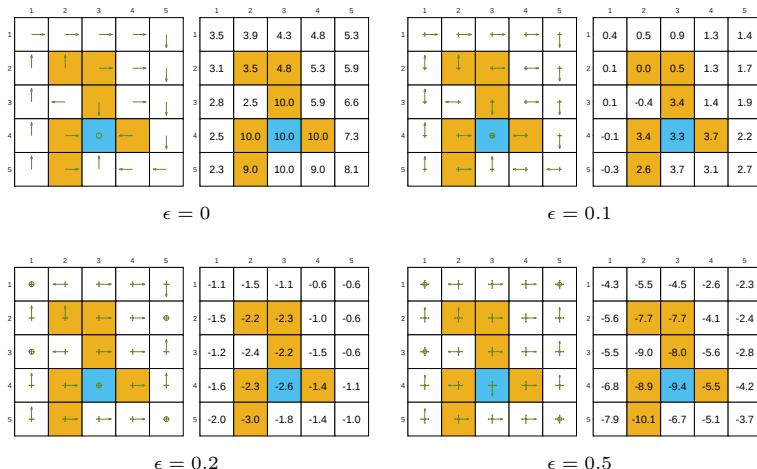
▷ 最优 ϵ -贪婪策略与贪婪最优策略并不一致！为什么？以目标状态为例思考。

▷ 示例：找到最优 ϵ -贪婪策略及其状态价值。



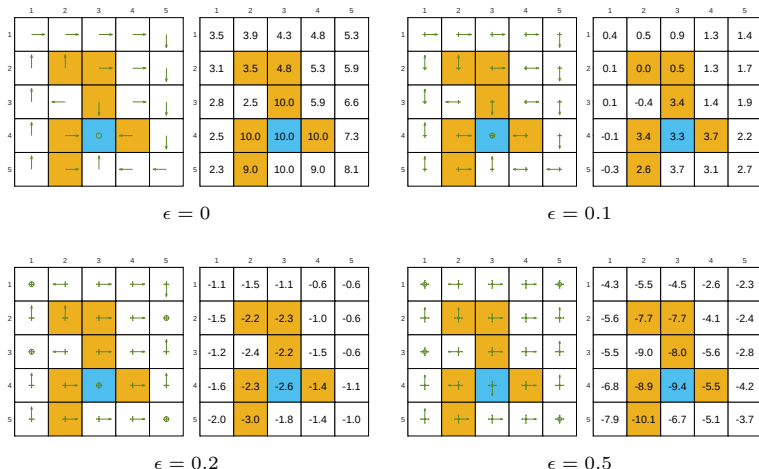
▷ 最优 ϵ -贪婪策略与贪婪最优策略并不一致！为什么？以目标状态为例思考。

▷ 示例：找到最优 ϵ -贪婪策略及其状态价值。



▷ 最优 ϵ -贪婪策略与贪婪最优策略并不一致！为什么？以目标状态为例思考。

▷ 示例：找到最优 ϵ -贪婪策略及其状态价值。



▷ 最优 ϵ -贪婪策略与贪婪最优策略并不一致！为什么？以目标状态为例思考。

关键点:

- 蒙特卡洛方法进行均值估计
- 三种算法:
 - MC Basic
 - MC ES (探索性起始)
 - MC ϵ -贪婪
- 三种算法之间的关系
- ϵ -贪婪策略的最优性与探索性